DASE7506 Advanced Machine Learning Methods

Mini Project 1 Report

Student ID: 3036706521

1 Introduction

The goal of this project is to improve the language modelling performance of the provided small GPT baseline on WikiText-2, with bits per byte (BPB) as the main evaluation metric. The baseline is a four-layer Transformer with a model width of 128 and a standard GELU feed-forward network. After 1,200 training steps, the locally reproduced baseline achieves a validation BPB of 2.0711.

The first modification replaces the original GELU feed-forward network with a parameter-matched SwiGLU block, allowing the architectural change to be evaluated without substantially increasing the model size. Further experiments explore longer training, increased model capacity, and dropout regularization. After stronger individual models are obtained, post-training checkpoint averaging and prediction-level ensembling are investigated, followed by a small temperature adjustment to the ensemble probabilities. The final predictor achieves a validation BPB of 1.5559 and a full-test BPB of 1.5775.

2 Experimental Setup

2.1 Evaluation Protocol

All experiments use the `7506-mp1-wt2-v2` protocol on the WikiText-2 raw-text dataset. The tokenizer is a BPE tokenizer fitted on the training split with a vocabulary size of 2048. The context length is fixed at 256 tokens, and evaluation is performed over independent causal windows.

Model quality is measured using bits per byte (BPB), with lower values indicating better performance. BPB is computed from the negative log-likelihood of the predicted tokens and normalized by the number of UTF-8 bytes in the corresponding text. The validation set contains 376,599 prediction targets and 1,148,007 UTF-8 bytes, while the test set contains 428,405 targets and 1,292,013 bytes.

During development, model and hyperparameter choices are based on validation BPB. The final test evaluation was performed only after the predictor configuration had been fixed. All official scoring runs use CPU FP32 inference. The final submission is also subject to limits on CPU scoring time, peak evaluation memory, and total inference asset size; these measurements are reported in Section 4.

For controlled training comparisons, models trained for the same number of steps process the same number of training targets. This is used explicitly in the parameter-matched SwiGLU comparison and the dropout ablation discussed later.

2.2 Baseline Model

The provided baseline is a small GPT-style causal Transformer with four Transformer blocks, a model width of 128, and four attention heads. Each feed-forward block uses a hidden dimension of 512 with GELU activation:

$$128 \rightarrow 512 \rightarrow 128.$$

The complete model contains 1,088,256 parameters. Using seed 17, the baseline was trained for 1,200 steps, corresponding to 9,830,400 processed training targets. The baseline was reproduced locally under the fixed evaluation protocol and achieved a validation BPB of 2.071087.

Table 1 summarizes the baseline configuration.

Table 1: Baseline model configuration and validation result.

Setting	Value
Model width	128
Attention heads	4
Transformer depth	4
MLP hidden dimension	512
Activation	GELU
Parameters	1,088,256
Training steps	1,200
Processed training targets	9,830,400
Random seed	17
Validation BPB	2.071087

3 Experiments and Ablations

Figure 1 summarizes the main validation performance progression across the experiments. Only configurations that form the main development path toward the final predictor are included; lower BPB indicates better performance.

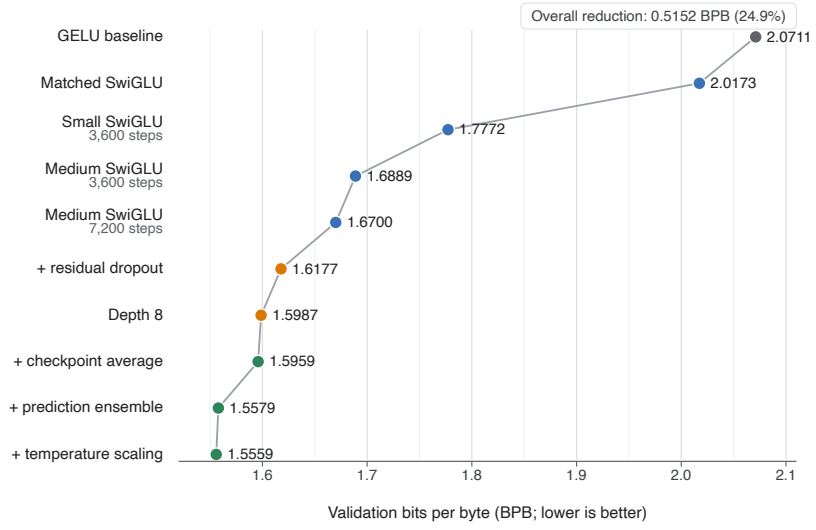


Figure 1: Validation BPB across the main experimental progression. Lower is better.

3.1 Parameter-Matched SwiGLU

The baseline uses a standard two-layer GELU MLP as its feed-forward block. SwiGLU was chosen as the first architectural modification because it introduces a gated transformation while leaving the attention mechanism, context length, and other parts of the Transformer unchanged. This makes it suitable for testing a localized change to the feed-forward computation.

Instead of applying a single nonlinear transformation to the expanded hidden representation, SwiGLU uses a value projection and a SiLU-activated gate:

$$\mathrm{SwiGLU}(x) = W_o \left[(W_v x) \odot \mathrm{SiLU}(W_g x) \right], \tag{1}$$

where W_v and W_g are the value and gate projections, respectively, W_o is the output projection, and $\odot$ denotes element-wise multiplication.

Using the same hidden dimension as the baseline would increase the parameter count because SwiGLU contains three linear projections instead of two. To approximately match the parameter budget, consider a baseline MLP with hidden dimension $4w$, whose two main weight matrices contain approximately

$$2w(4w) = 8w^2 \tag{2}$$

parameters. A SwiGLU block with hidden dimension h contains approximately

$$3wh \tag{3}$$

parameters. Matching the two gives

$$h \approx \frac{8w}{3}, \tag{4}$$

and the implementation uses

$$h = \left\lfloor \frac{8w}{3} \right\rfloor. \tag{5}$$

For $w = 128$, this gives $h = 341$. The resulting model contains 1,088,424 parameters, compared with 1,088,256 in the GELU baseline. Both models were trained for 1,200 steps with seed 17 and 9,830,400 processed training targets.

Under this matched setting, validation BPB improved from 2.071087 to 2.017312, an absolute reduction of 0.05378. This supports the effectiveness of the complete SwiGLU replacement in this setting without attributing the gain to a larger parameter budget.

3.2 Training and Capacity Scaling

After the parameter-matched SwiGLU model improved on the baseline, the next question was whether performance was still limited by training duration or by model capacity.

The small SwiGLU model was first trained for 3,600 rather than 1,200 steps while keeping its architecture unchanged. Validation BPB decreased from 2.017312 to 1.777224, showing that the shorter training budget had not yet exhausted the model.

Model capacity was then increased while keeping the training duration at 3,600 steps. The medium configuration uses width 192, depth 6, six attention heads, and a SwiGLU hidden dimension of 512, giving 3,113,472 parameters. At the same 3,600-step budget, validation BPB improved from 1.777224 for the small model to 1.688878 for the medium model. Since width, depth, attention heads, and feed-forward dimension all change together, this result should be interpreted as an effect of increased overall capacity rather than any single architectural factor.

A separate medium-model run was trained from scratch for 7,200 steps, reaching a final validation BPB of 1.670025. The improvement over the 3,600-step result was much smaller, and the best observed BPB was 1.668282 at step 6,000. This late-stage behaviour indicates a plateau with slight validation degradation, suggesting limited benefit from increasing the training budget further for this configuration.

3.3 Weight Averaging

The limited improvement from increasing the medium-model training budget to 7,200 steps suggested that further training alone was unlikely to provide large gains. Weight averaging was therefore tested as a post-training method that requires no additional optimization.

The 3,600-step and 7,200-step medium models were produced by separate training runs from scratch, but share the same architecture, seed, and training data. Their parameters were interpolated as

$$\theta_{\text{avg}} = \alpha\theta_{7200} + (1 - \alpha)\theta_{3600}, \tag{6}$$

where α is the contribution of the 7,200-step model.

The best result was obtained with $\alpha = 0.85$, reducing validation BPB from 1.670025 for the 7,200-step model to 1.658026. This shows that parameter interpolation between these two independently trained but structurally identical models can provide a small post-training improvement.

Unlike prediction ensembling, weight averaging produces a single set of parameters and therefore does not require multiple forward passes during inference. However, the gain was modest compared with the earlier improvements from training and capacity scaling.

3.4 Dropout Ablation

The late-stage plateau of the 7,200-step medium model suggested that regularization might improve generalization. To test this directly, residual dropout of 0.1 was added to the attention and MLP residual branches while keeping the model architecture, parameter count, training duration, seed, and processed training targets unchanged.

Both models contain 3,113,472 parameters and were trained for 7,200 steps. The no-dropout model achieved a final validation BPB of 1.670025, whereas the dropout model achieved 1.617746, an absolute improvement of approximately 0.05228 BPB.

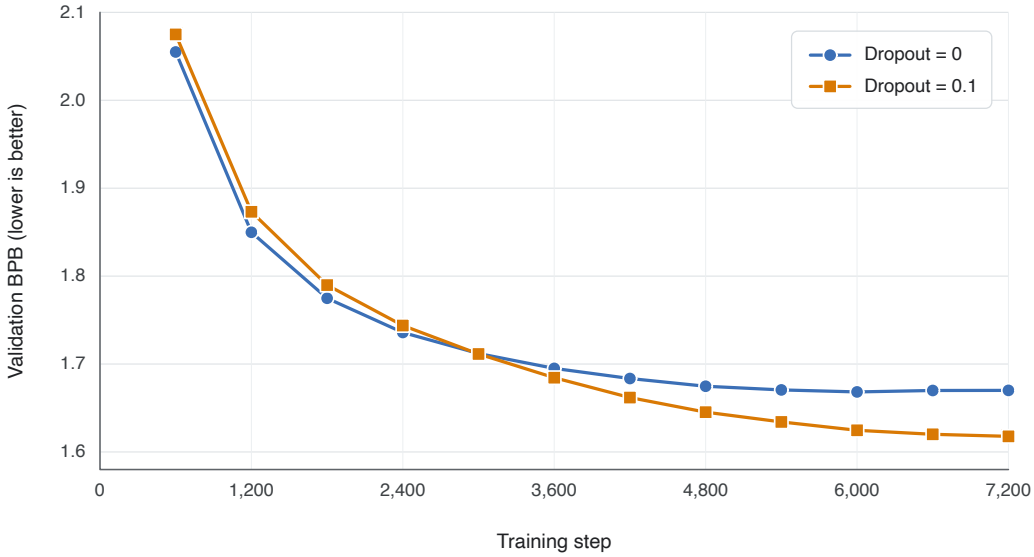


Figure 2: Validation BPB during training with and without residual dropout for the depth-6 medium SwiGLU model. Lower is better.

As shown in Figure 2, the two runs behave similarly early in training, but begin to diverge after roughly 3,000–3,600 steps. The no-dropout model reaches its best observed validation BPB at step 6,000 and then degrades slightly, while the dropout model continues to improve through step 7,200. Under this controlled setting, dropout therefore provides a clear improvement in validation performance and reduces the late-stage degradation observed without regularization.

The dropout configuration was retained for the subsequent depth comparison because it improved performance without increasing the number of trainable parameters.

3.5 Depth Scaling and Checkpoint Averaging

With dropout retained, a final capacity experiment increased the Transformer depth from 6 to 8 while keeping width 192, six attention heads, dropout 0.1, 7,200 training steps, and seed 17 unchanged. The parameter count increased from 3,113,472 to 4,003,712.

The depth-8 model reached a validation BPB of 1.598726, compared with 1.617746 for the depth-6 model. In this matched single-seed comparison, additional depth improved validation BPB under the regularized training setup, although the gain was smaller than that obtained from adding dropout. Training time also increased from approximately 3,250 seconds to 4,610 seconds, making further depth scaling less attractive.

The depth-8 run saved several late-stage checkpoints, allowing a lower-cost refinement through parameter averaging. Unlike the earlier weight-averaging experiment, these checkpoints came from the same continuous training run. Table 2 summarizes the tested checkpoint combinations.

Table 2: Late-checkpoint averaging for the depth-8 model.

Averaged checkpoints	Validation BPB
6000 + 7200	1.597224
6600 + 7200	1.596270
6000 + 6600 + 7200	**1.595879**
5400 + 6000 + 6600 + 7200	1.596544

Equal averaging of the step-6,000, 6,600, and 7,200 checkpoints gave the best result, improving validation BPB from 1.598726 to 1.595879. Including the earlier step-5,400 checkpoint slightly worsened performance, so the three-checkpoint average was retained.

A depth-10 model was not trained because the additional training cost was substantial while the expected marginal gain was uncertain.

3.6 Ensemble and Calibration

The depth-6 dropout model and the averaged depth-8 model were both strong predictors, but their different architectures prevent direct parameter averaging. Instead, their output probabilities were combined at inference time. For each next-token prediction, the ensemble distribution was defined as

$$p_{\text{ens}} = (1 - w)p_{d6} + wp_{d8}, \tag{7}$$

where p_{d6} is the probability distribution produced by the depth-6 model, p_{d8} is produced by the averaged depth-8 model, and w controls the contribution of the deeper model.

Several values of w were evaluated on the validation set. The results are shown in Table 3.

Table 3: Validation BPB for different prediction-ensemble weights.

Depth-8 weight w	Validation BPB
0.40	1.561400
0.50	1.558371
0.60	**1.557923**
0.70	1.560149
0.85	1.569704
0.95	1.582960

The best mixture assigned 60% of the probability mass to the depth-8 model and 40% to the depth-6 model, reducing validation BPB from 1.595879 for the stronger individual depth-8 predictor to 1.557923. The relatively large improvement compared with either individual model suggests that the two models may make partly complementary prediction errors. A plausible explanation for prediction ensembling being more effective here than further parameter averaging is that the two architectures can retain different useful predictions without requiring their parameters to occupy a compatible region. This interpretation remains tentative because token-level error overlap was not measured.

The ensemble improved prediction quality substantially, but it also requires both models to be evaluated during inference. Its additional computational cost was therefore measured explicitly before the final submission and is discussed in Section 4.

As a final validation-stage refinement, temperature scaling was applied to the ensemble distribution. Starting from the ensemble log probabilities, the adjusted distribution is

$$\log p_T = \log \operatorname{softmax}\left(\frac{\log p_{\text{ens}}}{T}\right), \tag{8}$$

where T controls the sharpness of the predicted distribution. Values close to 1 were tested because the aim was to adjust the confidence of the already strong ensemble rather than substantially

change its predictions.

Table 4: Validation BPB for temperature scaling of the final ensemble.

Temperature T	Validation BPB
0.95	1.564867
1.00	1.557923
1.05	**1.555891**
1.10	1.558022

A temperature of $T = 1.05$ produced the best validation result, improving BPB from 1.557923 to 1.555891. Since a temperature slightly above 1 softens the output distribution, this result is consistent with the uncalibrated ensemble being slightly over-confident on the validation set. The effect was small, so temperature scaling is best interpreted as a final calibration step rather than a major source of the overall performance gain.

The final predictor was therefore fixed as the 40/60 probability ensemble of the depth-6 dropout model and the averaged depth-8 model, followed by temperature scaling with $T = 1.05$. All ensemble weights and the temperature were selected using validation BPB before the full-test evaluation.

4 Final Evaluation

4.1 Final Predictor

The final predictor combines the depth-6 dropout model with the late-checkpoint-averaged depth-8 model. Their output probabilities are combined with weights 0.40 and 0.60, respectively, followed by temperature scaling with $T = 1.05$.

The complete predictor achieved a validation BPB of 1.555891. After all validation-stage choices had been fixed, the frozen final predictor was evaluated once on the full test set, producing a BPB of 1.577497 and a token perplexity of 27.05.

4.2 Resource Usage and Trade-off

The prediction ensemble improves BPB substantially over either individual model, but requires both models to be evaluated during inference. Its computational cost was therefore measured against the original baseline under CPU FP32 evaluation. The results are reported in Table 5.

Table 5: Final predictor performance and resource usage.

Metric	Baseline	Final predictor	Limit
Validation BPB	2.071087	1.555891	–
Full-test BPB	–	1.577497	–
Median validation CPU time	3.608 s	13.306 s	$\leq 5\times$ baseline
CPU time ratio	1.00×	3.69×	$\leq 5\times$
Peak evaluation RAM	–	1.71 GiB	≤ 4 GiB
Inference assets	–	27.23 MiB	≤ 64 MiB

The final predictor remains within all submission constraints. Its main cost is inference time, which increases to 3.69 times the baseline. Peak memory usage and inference asset size remain well below their limits. The full-test scoring time was 14.854 seconds.

5 Discussion and Limitations

5.1 Evidence and Attribution

Not all experiments support equally specific conclusions. Table 6 summarizes the main comparisons and their interpretation.

Table 6: Interpretation of the main experimental comparisons.

Comparison	Matched factors	Main change	Supported conclusion
GELU vs. matched SwiGLU	Nearly matched params, steps, seed, targets	FFN structure	SwiGLU improves validation BPB in this setting
D6 dropout 0 vs. 0.1	Architecture, params, steps, seed, targets	Residual dropout	Dropout improves validation BPB in this setting
Small vs. medium @ 3600	Steps, targets	Multiple capacity dimensions	Larger capacity improves BPB in this setting
D6 vs. D8 dropout	Width, heads, dropout, steps, seed	Depth	Additional depth improves BPB in this setting

The first two comparisons are the cleanest ablations because only one main design factor changes while the main training budget is held fixed. In contrast, the small-to-medium comparison changes several capacity dimensions simultaneously, so its gain should be attributed to increased overall capacity rather than to width, depth, or attention heads individually.

5.2 Failed Weight Averaging

Parameter averaging was not consistently effective. In particular, directly averaging the no-dropout and dropout models failed to improve on the original dropout model. Table 7 summarizes this unsuccessful cross-run averaging experiment.

Table 7: Cross-run averaging between no-dropout and dropout models.

Dropout-model weight	Validation BPB
0.80	1.733905
0.90	1.636545
0.95	1.618491
Original dropout model	**1.617746**

Although performance improves as more weight is assigned to the dropout model, none of the interpolated models outperform the original dropout checkpoint. This contrasts with the successful averaging of compatible medium models and late checkpoints from the same depth-8 run, suggesting that parameter compatibility matters and that weight averaging cannot be assumed to work for arbitrary model pairs.

5.3 Limitations

Most experiments use a single random seed, so robustness across different initializations was not measured. The search was also not exhaustive. A depth-10 model was considered but not trained because the additional training cost was high while its marginal benefit was uncertain.

The slight late-stage degradation of the no-dropout model is consistent with mild overfitting or validation fluctuation, but does not establish severe overfitting. Similarly, the strong improvement from prediction ensembling suggests complementary predictive behaviour between the depth-6 and depth-8 models, but this was not analyzed at the token-error level.

The final conclusions should therefore be interpreted within the tested configurations and resource budget rather than as evidence that the selected predictor is globally optimal.

6 Reproducibility

6.1 Experimental Environment

All experiments followed the `7506-mp1-wt2-v2` protocol. The main runs used random seed 17, a BPE vocabulary of 2048, and a context length of 256. Training and official evaluation were performed on CPU in FP32 using four PyTorch threads. Table 8 summarizes the main software and hardware environment.

Training used batches of 32 randomly sampled contiguous sequences, each containing 256 prediction targets. Optimization used AdamW with a base learning rate of 0.001, weight decay of 0.1, a 100-step linear warm-up, and cosine decay to 10% of the base rate. The global gradient norm was clipped to 1.0. For the longer runs with validation trajectories, validation BPB was evaluated every 600 steps.

Table 8: Main software and hardware environment.

Item	Setting
Python	3.12.14
PyTorch	2.7.1
Hardware	Apple M5 MacBook Air, 16 GB memory
Training / evaluation	CPU FP32, 4 threads
Random seed	17
Vocabulary size	2048
Context length	256

The major training logs consistently record the same PyTorch version, device, numerical precision, and thread count.

6.2 Training and Search Cost

Seven major training runs were used during model development, processing a cumulative 255.59 million training targets. Their recorded training time, excluding intermediate validation, was 13,244.7 seconds, or approximately 3.68 hours on CPU.

Post-training model selection also involved several validation-only searches. Table 9 summarizes the recorded cost.

Table 9: Recorded training and validation-search cost.

Component	Scale	Recorded time
Major training runs	7 runs	13,244.7 s (3.68 h)
Periodic validation during training	7 runs	419.1 s
Post-training validation search	22 distinct candidates	198.9 s

The post-training searches covered checkpoint interpolation, cross-run weight averaging, late-checkpoint averaging, ensemble weights, and temperature scaling. Their recorded scorer time does not include checkpoint construction, manual experiment orchestration, or development time, which were not logged and are therefore not estimated.

6.3 AI Assistance

AI assistance was used substantively for experiment planning, code-snippet suggestions, debugging, verification of logs and numerical consistency, generation of report figures, and organization and language editing of the report. All AI-assisted code and written material were reviewed and validated by the author. The implementation, experimental outputs, figures, and reported numerical results were checked against the actual code and run records before inclusion. The author remained responsible for all experimental decisions, interpretations, and final submitted content.

7 Conclusion

This project improved the provided small GPT baseline on WikiText-2 through a sequence of controlled architectural, training, and post-training experiments. A parameter-matched SwiGLU replacement provided an initial improvement over the GELU baseline, while longer training and increased model capacity produced larger gains. Residual dropout was particularly effective under the medium-model setting, and additional depth provided a further improvement. Checkpoint averaging, prediction ensembling, and temperature scaling were then used to refine the strongest models without additional full training runs.

The final predictor combines the depth-6 and depth-8 dropout models through probability ensembling followed by temperature scaling. It achieved a validation BPB of 1.555891 and a full-test BPB of 1.577497, while remaining within the required inference-time, memory, and asset size limits. The experiments also show that improvements cannot be attributed to a single modification: the final result comes from the combined effects of model architecture, training budget, regularization, model capacity, and post-training combination.